Національний технічний університет України
«Київський політехнічний інститут ім. І. Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Архітектура комп’ютерів
Лабораторна робота №4

«Розроблення модулів Linux Kernel (частина 1)»

Виконав:
Савич В. Я.
Залікова книжка №4212
Викладач

Київ 2026р.

Лабораторна робота №4

Тема: Розроблення модулів Linux Kernel (частина 1).

Мета: Ознайомитися з процесом створення, компіляції, завантаження та видалення модулів ядра Linux у системі зі спільною пам’яттю (на прикладі емулятора QEMU).

Хід роботи

Створення модуля ядра

Було створено два файли:

hello.c — вихідний код модуля

Makefile — для компіляції модуля під ядро ARM

hello.c

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/errno.h>
#include <linux/slab.h>
#include <linux/list.h>
#include <linux/ktime.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Ваше Ім'я");
MODULE_DESCRIPTION("Hello world module with list and time tracking");
MODULE_VERSION("1.2");

static uint times = 1;
module_param(times, uint, 0444);
MODULE_PARM_DESC(times, "Кількість виведених повідомлень 'Hello, world!' (uint)");

// Структура для зберігання часу виклику
```

```
struct hello_item {
    struct list_head list;
    ktime_t timestamp;
};

static LIST_HEAD(hello_list); // Статична голова списку

static int __init hello_init(void)
{
    int i;
    struct hello_item *item;

    if (times == 0 || (times >= 5 && times <= 10)) {
        pr_warn("hello: Warning! times=%u граничне значення, але продовжую\n", times);
    }

    if (times > 10) {
        pr_err("hello: Error! times=%u > 10 модуль не буде завантажено\n", times);
        return -EINVAL;
    }

    for (i = 0; i < times; i++) {
        item = kmalloc(sizeof(*item), GFP_KERNEL);
        if (!item) {
            pr_err("hello: Помилка виділення пам'яті\n");
            return -ENOMEM;
        }
        item->timestamp = ktime_get();
        list_add_tail(&item->list, &hello_list);
        pr_info("Hello, world! (%d)\n", i + 1);
    }
```

```
    return 0;
}

static void __exit hello_exit(void)
{
    struct hello_item *item, *tmp;

    pr_info("Goodbye, world! Вивід часу подій: \n");
    list_for_each_entry_safe(item, tmp, &hello_list, list) {
        pr_info(" time(ns): %lld\n", ktime_to_ns(item->timestamp));
        list_del(&item->list);
        kfree(item);
    }
}

module_init(hello_init);
module_exit(hello_exit);
```

Makefile

```
obj-m := hello.o

all:
	make -C $(HOME)/repos/linux-stable M=$(PWD) ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- modules

clean:
	make -C $(HOME)/repos/linux-stable M=$(PWD) clean
```

Компіляція модуля

```
Viacheslav@Lin3:~/Desktop/hello_module$ make
make -C /home/Viacheslav/repos/linux-stable M=/home/Viacheslav/Desktop/hello_module ARCH=arm CROSS_COMPIL
bihf- modules
  CC [M]  /home/Viacheslav/Desktop/hello_module/hello.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/Viacheslav/Desktop/hello_module/hello.mod.o
  LD [M]  /home/Viacheslav/Desktop/hello_module/hello.ko
Viacheslav@Lin3:~/Desktop/hello_module$ ls -l hello.ko
-rw-rw-r-- 1 Viacheslav Viacheslav 5628 Oct 28 13:41 hello.ko
```

Додавання модуля до rootfs

cd ~/repos/busybox

mkdir rootfs && cd rootfs

gunzip -c ../rootfs.cpio.gz | cpio –idmv

cp ~/Desktop/hello_module/hello.ko ./root/

find . | cpio -o -H newc | gzip -9 > ../rootfs_new.cpio.gz

cd ..

rm -rf rootfs

Запуск QEMU

qemu-system-arm -kernel install/boot/zImage -initrd rootfs_new.cpio.gz \

-machine virt -nographic -m 512 \

--append "root=/dev/ram0 rw console=ttyAMA0,115200 mem=512M"

Перевірка роботи

```
Viacheslav@Lin3: ~/repos/busybox — qemu-system-arm -kernel install/boot/zImage -initrd rootfs_new...
~/repos/busybox

Please press Enter to activate this console.
/ # cd /root
/root # insmod hello.ko
[   37.719811] hello: loading out-of-tree module taints kernel.
[   37.730554] Hello, world! (1)
/root # cat /sys/module/hello/parameters/times
1
/root # rmmod hello
[   54.595665] Goodbye, world! Вивід часу подій:
[   54.595991]  time(ns): 37631292064
/root # insmod hello.ko times=0
[   66.416723] hello: Warning! times=0 граничне значення, але продовжую
/root # rmmod hello
[   73.894569] Goodbye, world! Вивід часу подій:
/root # insmod hello.ko times=3
[   81.420493] Hello, world! (1)
[   81.420743] Hello, world! (2)
[   81.420893] Hello, world! (3)
/root # rmmod hello
[   89.003176] Goodbye, world! Вивід часу подій:
[   89.005423]  time(ns): 81321228400
[   89.006949]  time(ns): 81321501152
[   89.007071]  time(ns): 81321653088
/root # insmod hello.ko times=11
[  100.682320] hello: Error! times=11 > 10 модуль не буде завантажено
```

```
/root # mkdir -p /lib/modules/4.19.325
/root # touch /lib/modules/4.19.325/modules.dep
/root # modinfo hello.ko
filename:       hello.ko
author:         Ваше Ім'я
description:    Hello world module with list and time tracking
license:        GPL
parm:           times:Кількість виведених повідомлень 'Hello, world!' (uint)
version:        1.2
srcversion:     07E156607221DF43F00805F
depends:
vermagic:       4.19.325 SMP mod_unload ARMv7 p2v8
/root #
```

Висновок

Під час виконання лабораторної роботи було успішно розроблено, скомпільовано та досліджено власний модуль ядра Linux hello.ko для цільової архітектури ARM. Завдяки підготовці середовища та використанню Linux toolchain було здійснено успішну крос-компіляцію вихідного коду модуля із залученням заголовочних файлів зібраного ядра. У коді модуля реалізовано параметр times типу uint із описом для MODULE_PARM_DESC та логікою перевірки його граничних значень.

Під час виконання випробувань було підтверджено виведення системного попередження при значеннях times=0 та виникнення помилки -EINVAL із блокуванням завантаження при times > 10. Організація збереження точного часу подій ktime_t у зв'язаному списку list_head з використанням утиліт kmalloc і kfree дозволила коректно зафіксувати часові мітки та вивести їх у наносекундах під час вивантаження модуля.

Скомпільований модуль було додано до оновленого архіву файлової системи rootfs_new.cpio.gz та успішно протестовано у віртуальному середовищі емулятора QEMU. Перевірка команд insmod та rmmod показала стабільну роботу модуля з різними параметрами, а зчитування каталогу /sys/module/hello/parameters/times підтвердило правильність збереження значення в системі. Утиліта modinfo продемонструвала повний набір метаданих модуля, включаючи ліцензію, версію, автора та специфікацію вхідного параметра.